

INF-2204 Experiment project proposal

Ranked Distributions of Execution Time and Count across Programs and Platforms

Kathryn Ann Emblow
Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
kaemb0626@uit.no
TheKeons@github

Adrian Eriksen
Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
aderi2118@uit.no
kysck@github

Tuva Marie Eaton Johansen
Department of Computer Science
UiT The Arctic University of Norway
Tromsø, Norway
tujoh3800@uit.no
EarlGreyTT@github

I. INTRODUCTION

This report contains the early planning of our experiment.

A. Motivation and goal

A common rule of thumb in computer science is the 90-10 rule: 90% of the time is spent on 10% of the instructions. The origin of this rule is murky. The empirical evidence was initially presented [3] and later popularised by Knuth [4] as well as others, like Bentley [1]. However, the actual numbers used in the rule do not correspond to the evidence, and vary throughout papers and culture.

We aim to answer a broader question: what execution time and count distribution programs adhere to. We will measure and rank code locations by time and count. Will different types of software and hardware show the same distribution family and parameters?

B. Research questions

- 1) What distribution family and parameters result from ranking code locations by execution time and count, and do the rankings agree?
- 2) Is the distribution stable across hardware, OS, and runtime?
- 3) Is the distribution stable across different kinds of software and inputs?
- 4) Is the distribution stable across code granularity, such as instructions, lines, and functions?

We hypothesise a common distribution family with varying parameters across all software, inputs, platforms, and code granularities, and that rankings by time and count agree.

II. RELATED WORK

Our early literature review has yielded no results matching our exact research questions, as will be discussed further in section VI. The initial line of questioning regarded the 90-10 rule, leading to the aforementioned works by Donald Knuth [3], [4] and Jon Bentley [1]. These papers are well known and commonly cited in computer science, and were published by

recognized venues. Bentley quotes Knuth: “less than 4 per cent of a program generally accounts for more than half of its running time”, pertaining to his dataset of FORTRAN programs. Although Knuth mainly discusses compiler optimisation, his evidence comes from analysis of how these programs are run, which is relevant to our study. However, besides the popularisation of a 90-10 like rule, both Knuths papers and Bentleys Programming Pearls are otherwise of little relevance to our study.

In contrast to these older works, a study of the SPEC CPU2026 suite by Li et al. [5] discusses the most cutting-edge CPUs. This study has many figures to take inspiration from, and data that we can perhaps use to discuss our findings. However, it does not target our research questions, and has not been peer-reviewed as it is a preprint.

A profiling technique called Time-Proportional Instruction Profiling was introduced by Gottschall et al. [2]. They measured significant inaccuracy by typical profilers in the SPEC CPU2017 workload, caused by profilers’ struggle at accurately attributing time to instructions in out-of-order execution. As no hardware supports TIP, our research will concern itself with instruction time attribution errors. The paper was published at MICRO, a top-ranking venue (rated A* in CORE).

A point of interest for our study is the exact mathematical equations that describe any distribution we may find. Power laws in software study are discussed by P. Louridas et al. [6], finding that reuse of software, such as libraries and dependencies, account for most of the software used, in a power law distribution. This study appears to most closely resemble our research questions, however it discusses software in relation to each other, rather than in relation to itself. The power laws may still apply to the eventual distributions we may find. This study was published by ACM in Transactions on Software Engineering and Methodology, which has reached a score of A* in Core.

TABLE I
PRELIMINARY TABLE OF TEST PLATFORMS

Target	Platform		
	Hardware	OS	Runtime
ARM64 Linux	Raspberry Pi 5	Raspberry Pi OS	
ARM64 Mac OS	M3	macOS	
ARM64 Windows	Snapdragon X Plus	Windows	
x64 Linux	i5-10210U	Ubuntu	
x64 Linux	M3	macOS	QEMU Ubuntu
x64 Linux	Snapdragon X Plus	Windows	QEMU Ubuntu
x64 Linux	AMD Ryzen 7 6800HS	Ubuntu	
x64 Windows	AMD Ryzen 7 6800HS	Windows	
JVM bytecode	i5-10210U	Ubuntu	JVM
JVM bytecode	AMD Ryzen 7 6800HS	Windows	JVM
JVM bytecode	M3	macOS	JVM
JavaScript	i5-10210U	Ubuntu	V8 engine
JavaScript	AMD Ryzen 7 6800HS	Windows	V8 engine
JavaScript	M3	macOS	V8 engine
Python	i5-10210U	Ubuntu	CPython
Python	AMD Ryzen 7 6800HS	Windows	CPython
Python	M3	macOS	CPython

III. SYSTEM

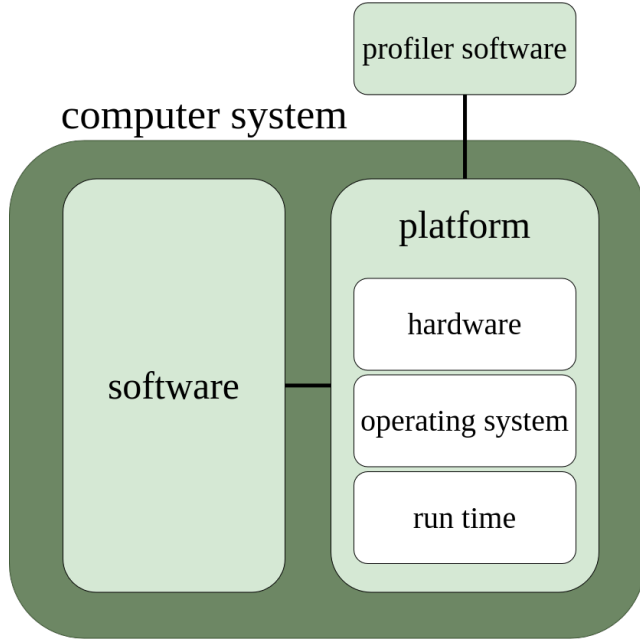


Fig. 1. Model of the system. Software refers to the chosen software for testing. Included in the platform are the relevant variables that may interact with each other, specified in Table 1.

Our general research questions yield a model of only two components: software and the platform they run on. A platform consists of hardware and any additional layers such as OS and runtime. The 1 figure shows the software and platform interacting, profiler by external profiler software. Profiling may be part of the platform and influence profiling results however.

Compiling configuration will influence the resulting software and its behaviour. The profiling will reveal the duration a platform spends at a given software's code location.

IV. EARLY ENVIRONMENT AND SCOPE

This project intends to test a broad variety of software since the project aims to figure out whether software shares a common ranked time-execution distribution. To target a variety of software, we intend to test software such as shell tools, browsers, cryptographic algorithms. We also aim to test branch-prevalant, CPU heavy, memory-intensive, loop and recursive-intensive, and highly optimised software. We are particularly interested in cryptographic algorithms, as their implementations usually protect against timing side-channel attacks.

Table I shows the preliminary set of platforms that will be tested at least in the early stages. There is a lot of hardware at the researchers' disposal to test. The table is not exhaustive, and may later be expanded with interesting combinations such as other runtimes, and the Ubuntu AMD Ryzen and Windows Snapdragon machines.

Native software could be profiled using Perf, Instruments and WPA. Valgrind or Clang's tools can be used to measure execution count. Software with runtimes can be profiled using the runtime's own profiling tools.

V. EARLY TIMELINE

- 1) Early literature review (now)
- 2) Choose profiler or testing program
- 3) Collect software to test
- 4) Preliminary testing stage
- 5) Check if the OS's have any effect

- 6) Research question specification
- 7) Testing stage
- 8) Data processing stage
- 9) Compare data to research question and hypothesis

VI. DISCUSSION

Due to a lack of previous research on our subject, the outcome is relatively uncertain as the 90-10 rule is more of a visualization than a rule. This means our research could show no consistent patterns in the distributions. However, that itself could make for an interesting find as it would disprove a rule of thumb that has been used for the majority of Computer Science history.

A more specific risk is that the platforms may require different profilers. These profiles might have different definitions on what 'time spent per instruction' might mean.

Also due to the parallel nature of modern systems, it might be difficult to actually measure the time per instruction [2]. This risk may become apparent when comparing smaller systems, such as the Raspberry Pi to larger more complex systems. MacBook's also have everything in one chip which might affect the latency.

VII. CONCLUSION

Measuring the distribution of time per instructions is a promising project despite little research done on the scope described in this report.

REFERENCES

- [1] Jon Bentley. "Programming pearls". In: *Commun. ACM* 28.9 (Sept. 1985), pp. 896–901. ISSN: 0001-0782. DOI: 10.1145/4284.315122. URL: <https://doi.org/10.1145/4284.315122>.
- [2] Björn Gottschall, Lieven Eeckhout, and Magnus Jahre. "TIP: Time-Proportional Instruction Profiling". In: *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. MICRO '21. Virtual Event, Greece: Association for Computing Machinery, 2021, pp. 15–27. ISBN: 9781450385572. DOI: 10.1145/3466752.3480058. URL: <https://doi.org/10.1145/3466752.3480058>.
- [3] Donald E. Knuth. "An empirical study of FORTRAN programs". In: *Software: Practice and Experience* 1.2 (1971), pp. 105–133. DOI: <https://doi.org/10.1002/spe.4380010203>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.4380010203>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.4380010203>.
- [4] Donald E. Knuth. "Structured Programming with go to Statements". In: *ACM Comput. Surv.* 6.4 (Dec. 1974), pp. 261–301. ISSN: 0360-0300. DOI: 10.1145/356635.356640. URL: <https://doi.org/10.1145/356635.356640>.
- [5] Ruihao Li et al. *SPEC CPU2026: Characterization, Representativeness, and Cross-Suite Comparison*. 2026. arXiv: 2605.03713 [cs.AR]. URL: <https://arxiv.org/abs/2605.03713>.
- [6] Panagiotis Louridas, Diomidis Spinellis, and Vasileios Vlachos. "Power laws in software". In: *ACM Trans. Softw. Eng. Methodol.* 18.1 (Oct. 2008). ISSN: 1049-331X. DOI: 10.1145/1391984.1391986. URL: <https://doi.org/10.1145/1391984.1391986>.